Coding & Solution

Phase 5 | Implemented application behavior

Frontend implementation

- App.jsx defines authenticated route gating and application navigation.
- Account supports registration, login, profile loading, and logout.
- CreateTrip and TripForm collect trip details and request generation.
- SavedTrips and TripView support list/detail management, updates, deletion, regeneration, and export actions.
- The API client attaches bearer tokens and maintains a per-user local-storage trip cache.

Backend implementation

- FastAPI configures CORS, runtime validation, authentication and trip routers, root response, and health endpoint.
- Pydantic schemas define account, trip, day, and activity payloads.
- Authentication service normalizes emails, hashes passwords, and verifies credentials.
- Trip services persist user-scoped trip and itinerary data through Supabase or process memory.
- AI orchestration supports Groq, Ollama, and a local curated/procedural fallback.
- History service records selected generation and trip-management actions.

Persistence and delivery

The Supabase schema defines users, trips, itineraries, preferences, saved places, and history with indexes and row-level security enabled. The repository also has Docker Compose, Render, and Vercel configuration. Deployment success is not asserted by this implementation report.

Important implementation caveats

- Hugging Face settings are present, but no provider adapter was found.
- Backend in-memory stores reset when the process restarts.
- Trip share URLs lead to authenticated routes.
- Strict start-date/end-date ordering should be added if required.

Prepared from the current project source and configuration. Product assumptions and proposed work are labeled; no user research, test result, or deployment result is implied unless explicitly stated.